

Advanced Python Programming

Unit 1 - Threading, Networking & Database | Assignment No. 2

Section A: Threading

Q1. Write a program to create Thread by using various methods.

Program:

```
import threading, time

# Method 1: Thread with target function
def greet(name):
    print(f"[M1] Hello {name}! in {threading.current_thread().name}")
    time.sleep(0.5)

t1 = threading.Thread(target=greet, args=("Alice",), name="T-Greet")
t1.start(); t1.join()

# Method 2: Subclassing Thread
class MyThread(threading.Thread):
    def __init__(self, task):
        super().__init__()
        self.task = task
    def run(self):
        print(f"[M2] Task: {self.task} in {threading.current_thread().name}")
        time.sleep(0.5)

t2 = MyThread("DB Backup"); t2.start(); t2.join()

# Method 3: Lambda target
t3 = threading.Thread(target=lambda: print("[M3] Lambda thread!"))
t3.start(); t3.join()

# Method 4: Multiple threads via list
def worker(n):
    print(f"[M4] Worker-{n} running"); time.sleep(0.3)
    print(f"[M4] Worker-{n} done")

threads = [threading.Thread(target=worker, args=(i,)) for i in range(1,4)]
[t.start() for t in threads]; [t.join() for t in threads]
print("All threads completed.")
```

Output:

```
[M1] Hello Alice! in T-Greet [M2] Task: DB Backup in Thread-1 [M3] Lambda thread! [M4] Worker-1
running [M4] Worker-2 running [M4] Worker-3 running [M4] Worker-1 done | Worker-2 done | Worker-3
done All threads completed.
```

Q2. Write a program on inter thread communication by using event.

Program:

```
import threading, time

event = threading.Event()

def producer():
    print("[Producer] Preparing data...")
    time.sleep(2)
    print("[Producer] Data ready! Setting event.")
    event.set()
```

```
def consumer():
    print("[Consumer] Waiting for data...")
    event.wait()          # blocks until event.set() is called
    print("[Consumer] Data received! Processing...")

t1 = threading.Thread(target=producer)
t2 = threading.Thread(target=consumer)
t2.start(); t1.start()
t1.join(); t2.join()
print("Event-based communication done.")
```

Output:

```
[Consumer] Waiting for data... [Producer] Preparing data... [Producer] Data ready! Setting event.
[Consumer] Data received! Processing... Event-based communication done.
```

Note: event.wait() blocks thread until event.set() is called by another thread.

Q3. Write a program on inter thread communication by using condition.

Program:

```
import threading, time

condition = threading.Condition()
shared_data = []

def producer():
    for i in range(1, 4):
        time.sleep(1)
        with condition:
            shared_data.append(f"Item-{i}")
            print(f"[Producer] Produced Item-{i}")
            condition.notify()

def consumer():
    for _ in range(3):
        with condition:
            while not shared_data:
                print("[Consumer] Waiting...")
                condition.wait()
            item = shared_data.pop(0)
            print(f"[Consumer] Consumed {item}")

t1 = threading.Thread(target=producer)
t2 = threading.Thread(target=consumer)
t1.start(); t2.start()
t1.join(); t2.join()
print("Condition-based communication done.")
```

Output:

```
[Consumer] Waiting... [Producer] Produced Item-1 [Consumer] Consumed Item-1 [Consumer] Waiting...
[Producer] Produced Item-2 [Consumer] Consumed Item-2 [Consumer] Waiting... [Producer] Produced
Item-3 [Consumer] Consumed Item-3 Condition-based communication done.
```

Note: condition.wait() releases lock and suspends thread until condition.notify() is called.

Q4. Write a program to demonstrate use of semaphore.

Program:

```
import threading, time

semaphore = threading.Semaphore(2)  # max 2 concurrent threads

def access_resource(tid):
```

```

print(f"Thread-{tid}: Waiting...")
with semaphore:
    print(f"Thread-{tid}: ACQUIRED - using resource")
    time.sleep(1.5)
    print(f"Thread-{tid}: RELEASED")

threads = [threading.Thread(target=access_resource, args=(i,))
            for i in range(1, 6)]
for t in threads: t.start()
for t in threads: t.join()
print("Semaphore demo complete.")

```

Output:

```

Thread-1: Waiting... Thread-2: Waiting... Thread-3: Waiting... Thread-1: ACQUIRED - using resource
Thread-2: ACQUIRED - using resource (Thread-3,4,5 wait for slot...) Thread-1: RELEASED Thread-3:
ACQUIRED - using resource ... Semaphore demo complete.

```

Note: Semaphore(2) allows max 2 threads concurrently. Others wait — like a DB connection pool.

Q5. Write a program to demonstrate the limitation of Lock.

Program:

```

import threading, time

lock = threading.Lock()
rlock = threading.RLock()

# Limitation 1: Deadlock (same thread re-acquires Lock)
def deadlock_risk():
    lock.acquire()
    print("First lock acquired.")
    # lock.acquire() # <-- Would DEADLOCK! Commented to prevent hang
    print("DEADLOCK avoided (second acquire skipped).")
    lock.release()

# Solution: RLock allows same thread to re-acquire
def rlock_safe():
    with rlock:
        print("RLock level-1 acquired")
        with rlock:      # Re-entrant - NO deadlock
            print("RLock level-2 acquired (safe!)")

# Limitation 2: No starvation prevention
counter = 0
def increment(name):
    global counter
    for _ in range(3):
        with lock:
            val = counter; time.sleep(0.01); counter = val + 1
            print(f" {name}: counter = {counter}")

print("=== Limitation 1: Deadlock Risk ===")
deadlock_risk()
print("\n=== Solution: RLock ===")
rlock_safe()
print("\n=== Limitation 2: No starvation control ===")
t1 = threading.Thread(target=increment, args=("T-A",))
t2 = threading.Thread(target=increment, args=("T-B",))
t1.start(); t2.start(); t1.join(); t2.join()
print(f"Final counter: {counter}")

```

Output:

```
=== Limitation 1: Deadlock Risk === First lock acquired. DEADLOCK avoided (second acquire skipped).
=== Solution: RLock === RLock level-1 acquired RLock level-2 acquired (safe!) === Limitation 2: No
starvation control === T-A: counter = 1 T-B: counter = 2 (order not guaranteed) T-A: counter = 3 ...
Final counter: 6
```

Note: Lock cannot be re-acquired by same thread (deadlock risk). Use RLock for re-entrant locking.

Q6. Write a program to create a thread by using parameterized function.

Program:

```
import threading, time

def print_squares(start, end, delay=0.3):
    name = threading.current_thread().name
    print(f"[{name}] Squares {start} to {end}:")
    for n in range(start, end + 1):
        print(f"    {n}^2 = {n*n}")
        time.sleep(delay)

def calculate_sum(numbers, label):
    print(f"[{label}] Sum of {numbers} = {sum(numbers)}")

# positional args
t1 = threading.Thread(target=print_squares, args=(1, 3),
                      name="SqThread-1")

# keyword args
t2 = threading.Thread(target=print_squares,
                      kwargs={"start": 4, "end": 6, "delay": 0.2},
                      name="SqThread-2")

# mixed
t3 = threading.Thread(target=calculate_sum,
                      args=(10, 20, 30),),
                      kwargs={"label": "SumThread"})

t1.start(); t2.start(); t3.start()
t1.join(); t2.join(); t3.join()
print("All parameterized threads finished.")
```

Output:

```
[SqThread-1] Squares 1 to 3: 1^2 = 1 [SqThread-2] Squares 4 to 6: 4^2 = 16 [SumThread] Sum of [10,
20, 30] = 60 2^2 = 4 | 5^2 = 25 3^2 = 9 | 6^2 = 36 All parameterized threads finished.
```

Note: Use args=() for positional, kwargs={} for keyword parameters in Thread constructor.

Q7. Write a program to demonstrate use of sleep method.

Program:

```
import threading, time

def task(name, sleep_time, steps):
    for i in range(1, steps + 1):
        print(f"[{name}] Step {i}/{steps} at {time.strftime('%H:%M:%S')}")
        time.sleep(sleep_time)
    print(f"[{name}] Done.")

def countdown(n):
    print(f"Countdown: ", end="")
    for s in range(n, 0, -1):
        print(f"{s}...", end=" ", flush=True)
        time.sleep(1)
    print("GO!")

t1 = threading.Thread(target=task, args=("FastTask", 0.5, 4))
```

```
t2 = threading.Thread(target=task, args=("SlowTask", 1.2, 2))
t3 = threading.Thread(target=countdown, args=(3,))

start = time.time()
t1.start(); t2.start(); t3.start()
t1.join(); t2.join(); t3.join()
print(f"Total time: {time.time()-start:.2f}s")
```

Output:

```
[FastTask] Step 1/4 at 10:30:00 [SlowTask] Step 1/2 at 10:30:00 Countdown: 3... 2... 1... GO!
[FastTask] Step 2/4 at 10:30:00 [FastTask] Step 3/4 at 10:30:01 [SlowTask] Step 2/2 at 10:30:01
[FastTask] Step 4/4 at 10:30:01 [FastTask] Done. [SlowTask] Done. Total time: 2.41s
```

Note: `time.sleep(n)` suspends only the calling thread, not other threads. Others run concurrently.

Q8. Write a program to demonstrate use of join method.

Program:

```
import threading, time

results = {}

def fetch(source, delay):
    print(f"[{source}] Fetching... ({delay}s)")
    time.sleep(delay)
    results[source] = f"Data from {source}"
    print(f"[{source}] Done.")

def process():
    print(f"Processing {len(results)} results:")
    for k, v in results.items():
        print(f" >> {v}")

t1 = threading.Thread(target=fetch, args=("API", 2.0))
t2 = threading.Thread(target=fetch, args=("DB", 1.0))
t3 = threading.Thread(target=fetch, args=("Cache", 0.5))

t1.start(); t2.start(); t3.start()
t1.join() # Wait for API
t2.join() # Wait for DB
t3.join() # Wait for Cache

print("\nAll fetches done (join guaranteed).")
process()

# join with timeout
t4 = threading.Thread(target=fetch, args=("SlowAPI", 5))
t4.start()
t4.join(timeout=2)
if t4.is_alive():
    print("SlowAPI timed out - continuing without it.")
```

Output:

```
[Cache] Fetching... (0.5s) [DB] Fetching... (1.0s) [API] Fetching... (2.0s) [Cache] Done. [DB] Done.
[API] Done. All fetches done (join guaranteed). Processing 3 results: >> Data from API >> Data from
DB >> Data from Cache [SlowAPI] Fetching... (5s) SlowAPI timed out - continuing without it.
```

Note: `join()` ensures main thread waits for worker to finish. `join(timeout=N)` adds a time limit.

Section B: MySQL Database Connectivity

Q9. Write a program to create connection with MySQL database.

Program:

```
import mysql.connector
from mysql.connector import Error

try:
    conn = mysql.connector.connect(
        host = "localhost",
        user = "root",
        password = "yourpassword" # Replace with your MySQL password
    )
    if conn.is_connected():
        info = conn.get_server_info()
        print(f"Connected to MySQL version: {info}")
        cursor = conn.cursor()
        cursor.execute("SELECT DATABASE();")
        print(f"Current Database: {cursor.fetchone()}")

except Error as e:
    print(f"Connection Error: {e}")

finally:
    if conn.is_connected():
        cursor.close()
        conn.close()
    print("Connection closed successfully.")
```

Output:

```
Connected to MySQL version: 8.0.32 Current Database: (None,) Connection closed successfully.
```

Note: Install: pip install mysql-connector-python. Replace 'yourpassword' with your MySQL password.

Q10. Write a program to create MySQL database 'college'.

Program:

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root", password="yourpassword"
)
cursor = conn.cursor()

cursor.execute("CREATE DATABASE IF NOT EXISTS college")
print("Database 'college' created!")

cursor.execute("SHOW DATABASES")
print("\nAll Databases:")
for db in cursor.fetchall():
    tag = " <-- NEW" if db[0] == "college" else ""
    print(f" {db[0]}{tag}")

cursor.execute("USE college")
print("\nNow using: college")

cursor.close(); conn.close()
```

Output:

```
Database 'college' created! All Databases: information_schema college <-- NEW mysql
performance_schema sys Now using: college
```

Q11. Write a program to create tables student, book, employee.

Program:

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college"
)
cursor = conn.cursor()

# Table 1: student
cursor.execute(
    "CREATE TABLE IF NOT EXISTS student ("
    " student_id INT PRIMARY KEY AUTO_INCREMENT,"
    " name VARCHAR(100) NOT NULL,"
    " branch VARCHAR(50),"
    " semester INT,"
    " percentage FLOAT,"
    " email VARCHAR(100) UNIQUE"
    ")"
)
print("Table 'student' created.")

# Table 2: book
cursor.execute(
    "CREATE TABLE IF NOT EXISTS book ("
    " book_id INT PRIMARY KEY AUTO_INCREMENT,"
    " title VARCHAR(200) NOT NULL,"
    " author VARCHAR(100),"
    " publisher VARCHAR(100),"
    " price DECIMAL(8,2),"
    " year_pub INT"
    ")"
)
print("Table 'book' created.")

# Table 3: employee
cursor.execute(
    "CREATE TABLE IF NOT EXISTS employee ("
    " emp_id INT PRIMARY KEY AUTO_INCREMENT,"
    " emp_name VARCHAR(100) NOT NULL,"
    " department VARCHAR(50),"
    " designation VARCHAR(50),"
    " salary DECIMAL(10,2),"
    " join_date DATE"
    ")"
)
print("Table 'employee' created.")

cursor.execute("SHOW TABLES")
print("\nTables in college DB:")
for t in cursor.fetchall():
    print(f" - {t[0]}")

conn.commit(); cursor.close(); conn.close()

```

Output:

```

Table 'student' created. Table 'book' created. Table 'employee' created. Tables in college DB: -
book - employee - student

```

Q12. Write a program to insert multiple records in table.

Program:

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college"
)
cursor = conn.cursor()

students = [
    ("Rahul Sharma", "CS", 5, 87.5, "rahul@college.edu"),
    ("Priya Patel", "Electronics", 3, 91.2, "priya@college.edu"),
    ("Amit Desai", "Mechanical", 7, 78.9, "amit@college.edu"),
    ("Sneha Kulkarni", "CS", 5, 82.3, "sneha@college.edu"),
    ("Rohan Joshi", "Civil", 1, 74.0, "rohan@college.edu"),
]
sql = ("INSERT INTO student (name,branch,semester,percentage,email)"
      " VALUES (%s,%s,%s,%s,%s)")
cursor.executemany(sql, students)
conn.commit()
print(f"{cursor.rowcount} student records inserted!")

employees = [
    ("Prof. Mehta", "CS Dept", "Professor", 85000.00, "2018-06-01"),
    ("Dr. Patil", "Electronics", "HOD", 120000.00, "2010-01-15"),
    ("Mrs. Sharma", "Admin", "Clerk", 35000.00, "2020-03-10"),
]
cursor.executemany(
    "INSERT INTO employee(emp_name,department,designation,salary,join_date)"
    " VALUES (%s,%s,%s,%s,%s)", employees)
conn.commit()
print(f"{cursor.rowcount} employee records inserted!")

cursor.execute("SELECT student_id, name, branch, percentage FROM student")
print(f"\n{'ID':<5}{'Name':<20}{'Branch':<15}{'%':<10}")
print("-"*50)
for row in cursor.fetchall():
    print(f"{row[0]:<5}{row[1]:<20}{row[2]:<15}{row[3]:<10}")

cursor.close(); conn.close()

```

Output:

```

5 student records inserted! 3 employee records inserted! ID Name Branch %
----- 1 Rahul Sharma CS 87.5 2 Priya Patel Electronics
91.2 3 Amit Desai Mechanical 78.9 4 Sneha Kulkarni CS 82.3 5 Rohan Joshi Civil 74.0

```

Note: executemany() inserts multiple rows in one call. Always conn.commit() after DML operations.

Q13. Write a program to Select and update records in table.

Program:

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college"
)
cursor = conn.cursor()

# SELECT all
print("=== All Students ===")
cursor.execute("SELECT name, branch, percentage FROM student")

```



```

for r in cursor.fetchall():
    print(f" {r[0]} | {r[1]} | {r[2]}%")

# SELECT with WHERE
print("\n=== CS students % > 80 ===")
cursor.execute(
    "SELECT name, percentage FROM student "
    "WHERE branch=%s AND percentage>%s", ("CS", 80))
for r in cursor.fetchall():
    print(f" {r[0]}: {r[1]}%")

# SELECT with ORDER BY
print("\n=== Top 3 by Percentage ===")
cursor.execute(
    "SELECT name, percentage FROM student "
    "ORDER BY percentage DESC LIMIT 3")
for i, r in enumerate(cursor.fetchall(), 1):
    print(f" Rank {i}: {r[0]} - {r[1]}%")

# UPDATE single record
cursor.execute(
    "UPDATE student SET percentage=%s WHERE name=%s",
    (95.0, "Priya Patel"))
conn.commit()
print(f"\nUpdated {cursor.rowcount} record(s).")

# UPDATE with arithmetic
cursor.execute(
    "UPDATE employee SET salary=salary*1.10 WHERE department=%s",
    ("CS Dept",))
conn.commit()
print(f"Salary hike applied to {cursor.rowcount} record(s).")

# Verify
cursor.execute(
    "SELECT name, percentage FROM student WHERE name=%s",
    ("Priya Patel",))
print(f"Priya new %: {cursor.fetchone()[1]}")

cursor.close(); conn.close()

```

Output:

```

=== All Students === Rahul Sharma | CS | 87.5% Priya Patel | Electronics | 91.2% ... === CS students
% > 80 === Rahul Sharma: 87.5% Sneha Kulkarni: 82.3% === Top 3 by Percentage === Rank 1: Priya Patel
- 91.2% Rank 2: Rahul Sharma - 87.5% Rank 3: Sneha Kulkarni - 82.3% Updated 1 record(s). Salary hike
applied to 1 record(s). Priya new %: 95.0

```

Q14. Write a program to delete specific record in table.

Program:

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college"
)
cursor = conn.cursor()

cursor.execute("SELECT COUNT(*) FROM student")
print(f"Before delete: {cursor.fetchone()[0]} records")

```

```

# Delete by primary key
cursor.execute("DELETE FROM student WHERE student_id=%s", (5,))
conn.commit()
print(f"Deleted {cursor.rowcount} record with ID=5")

# Delete by condition
cursor.execute("DELETE FROM student WHERE percentage<=76", (76.0,))
conn.commit()
print(f"Deleted {cursor.rowcount} student(s) with %<76")

cursor.execute("SELECT COUNT(*) FROM student")
print(f"After delete: {cursor.fetchone()[0]} records")

# Safe check before delete
email = "amit@college.edu"
cursor.execute("SELECT * FROM student WHERE email=%s", (email,))
rec = cursor.fetchone()
if rec:
    cursor.execute("DELETE FROM student WHERE email=%s", (email,))
    conn.commit()
    print(f"Deleted: {rec[1]}")
else:
    print(f"Not found: {email}")

# Show remaining
cursor.execute("SELECT student_id, name, percentage FROM student")
print("\nRemaining students:")
for r in cursor.fetchall():
    print(f" ID:{r[0]} | {r[1]} | {r[2]}%")

cursor.close(); conn.close()

```

Output:

```

Before delete: 5 records Deleted 1 record with ID=5 Deleted 1 student(s) with %<76 After delete: 3
records Deleted: Amit Desai Remaining students: ID:1 | Rahul Sharma | 87.5% ID:2 | Priya Patel |
95.0% ID:4 | Sneha Kulkarni | 82.3%

```

Note: Always check record exists before DELETE to avoid silent failures.

Q15. Write a program to call parameterized procedure.

Program:

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college"
)
cursor = conn.cursor()

# Create stored procedure with IN parameters
cursor.execute("DROP PROCEDURE IF EXISTS GetStudentByBranch")
create_proc = (
    "CREATE PROCEDURE GetStudentByBranch("
    " IN branch_name VARCHAR(50),"
    " IN min_pct FLOAT,"
    " OUT stu_count INT"
    ") "
    "BEGIN "
    " SELECT name, branch, percentage FROM student "
    " WHERE branch = branch_name AND percentage >= min_pct "

```

```

" ORDER BY percentage DESC; "
" SELECT COUNT(*) INTO stu_count FROM student "
" WHERE branch = branch_name AND percentage >= min_pct; "
"END"
)
cursor.execute(create_proc)
print("Procedure 'GetStudentByBranch' created.")

# Call the parameterized procedure
result_args = cursor.callproc("GetStudentByBranch",
                              ("CS", 80.0, 0))

print("\nCS Students with % >= 80:")
for res in cursor.stored_results():
    for row in res.fetchall():
        print(f" {row[0]} | {row[1]} | {row[2]}%")

print(f"\nOUT param - Count: {result_args[2]}")
conn.commit(); cursor.close(); conn.close()

```

Output:

```

Procedure 'GetStudentByBranch' created. CS Students with % >= 80: Priya Patel | CS | 95.0% Rahul Sharma | CS | 87.5% Sneha Kulkarni | CS | 82.3% OUT param - Count: 3

```

Note: callproc(name, args) calls procedure. stored_results() fetches SELECT output from procedure.

Q16. Write a program to retrieve a data by using procedure.

Program:

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college"
)
cursor = conn.cursor()

# Procedure to retrieve all employees
cursor.execute("DROP PROCEDURE IF EXISTS GetAllEmployees")
cursor.execute(
    "CREATE PROCEDURE GetAllEmployees() "
    "BEGIN "
    " SELECT emp_id, emp_name, department, salary "
    " FROM employee ORDER BY salary DESC; "
    "END"
)

# Procedure to get by department
cursor.execute("DROP PROCEDURE IF EXISTS GetEmpByDept")
cursor.execute(
    "CREATE PROCEDURE GetEmpByDept(IN dept VARCHAR(50)) "
    "BEGIN "
    " SELECT emp_name, designation, salary "
    " FROM employee WHERE department = dept; "
    "END"
)

print("Procedures created.")

# Retrieve all employees
print("\n=== All Employees ===")
cursor.callproc("GetAllEmployees")

```

```

for res in cursor.stored_results():
    for r in res.fetchall():
        print(f" ID:{r[0]} | {r[1]} | {r[2]} | Rs.{r[3]:.2f}")

# Retrieve by department
print("\n=== Electronics Dept ===")
cursor.callproc("GetEmpByDept", ("Electronics",))
for res in cursor.stored_results():
    for r in res.fetchall():
        print(f" {r[0]} | {r[1]} | Rs.{r[2]:.2f}")

cursor.close(); conn.close()

```

Output:

```

Procedures created. === All Employees === ID:2 | Dr. Patil | Electronics | Rs.120000.00 ID:1 | Prof.
Mehta | CS Dept | Rs.93500.00 ID:3 | Mrs. Sharma | Admin | Rs.35000.00 === Electronics Dept === Dr.
Patil | HOD | Rs.120000.00

```

Q17. Write a program to update a data by using procedure.

Program:

```

import mysql.connector

conn = mysql.connector.connect(
    host="localhost", user="root",
    password="yourpassword", database="college"
)
cursor = conn.cursor()

# Update procedure with IN/OUT params
cursor.execute("DROP PROCEDURE IF EXISTS UpdateStudentMarks")
cursor.execute(
    "CREATE PROCEDURE UpdateStudentMarks("
    " IN s_id INT, IN new_pct FLOAT,"
    " OUT old_pct FLOAT, OUT rows_aff INT"
    ") "
    "BEGIN "
    " SELECT percentage INTO old_pct "
    " FROM student WHERE student_id = s_id; "
    " UPDATE student SET percentage = new_pct "
    " WHERE student_id = s_id; "
    " SET rows_aff = ROW_COUNT(); "
    "END"
)

# Procedure for department salary hike
cursor.execute("DROP PROCEDURE IF EXISTS UpdateSalaryByDept")
cursor.execute(
    "CREATE PROCEDURE UpdateSalaryByDept("
    " IN dept VARCHAR(50), IN hike FLOAT"
    ") "
    "BEGIN "
    " UPDATE employee "
    " SET salary = salary + (salary * hike / 100) "
    " WHERE department = dept; "
    " SELECT emp_name, salary AS new_salary "
    " FROM employee WHERE department = dept; "
    "END"
)
print("Update procedures created.")

```

```
# Call update student marks
result = cursor.callproc("UpdateStudentMarks", (1, 92.0, 0.0, 0))
conn.commit()
print(f"\nStudent ID 1: Old%={result[2]}, New%={result[1]}, Rows={result[3]}")

# Call salary update
print("\n=== Admin Dept 15% Hike ===")
cursor.callproc("UpdateSalaryByDept", ("Admin", 15.0))
conn.commit()
for res in cursor.stored_results():
    for r in res.fetchall():
        print(f" {r[0]} | New Salary: Rs.{r[1]:.2f}")

cursor.close(); conn.close()
```

Output:

```
Update procedures created. Student ID 1: Old%=87.5, New%=92.0, Rows=1 === Admin Dept 15% Hike ===
Mrs. Sharma | New Salary: Rs.40250.00
```

Note: OUT parameters return values from stored procedure. Placeholder 0 / 0.0 used for OUT params.

Section C: Networking in Python

Q18. Write a program to create TCP client socket.

Program:

```
import socket

def tcp_client():
    HOST = '127.0.0.1'
    PORT = 65432

    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        client.connect((HOST, PORT))
        print(f"Connected to {HOST}:{PORT}")

        messages = ["Hello Server!", "How are you?", "quit"]
        for msg in messages:
            client.sendall(msg.encode('utf-8'))
            print(f"[Client] Sent: {msg}")
            if msg == "quit":
                break
            data = client.recv(1024)
            print(f"[Client] Received: {data.decode('utf-8')}")

    except ConnectionRefusedError:
        print("Connection refused! Start server first.")
    finally:
        client.close()
        print("[Client] Socket closed.")

tcp_client()
```

Output:

```
Connected to 127.0.0.1:65432 [Client] Sent: Hello Server! [Client] Received: Echo: Hello Server!
[Client] Sent: How are you? [Client] Received: Echo: How are you? [Client] Sent: quit [Client]
Socket closed.
```

Note: Run Q19 (Server) first. AF_INET = IPv4, SOCK_STREAM = TCP connection-oriented protocol.

Q19. Write a program to create TCP Server socket.

Program:

```
import socket, threading

def handle_client(conn, addr):
    print(f"[Server] Connection from {addr}")
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            msg = data.decode('utf-8')
            print(f"[Server] From {addr}: {msg}")
            if msg.lower() == "quit":
                print(f"[Server] {addr} disconnected.")
                break
            conn.sendall(f"Echo: {msg}".encode('utf-8'))

def tcp_server():
    HOST, PORT = '127.0.0.1', 65432
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind((HOST, PORT))
        srv.listen(5)
        print(f"[Server] Listening on {HOST}:{PORT}")
        while True:
            conn, addr = srv.accept()
            t = threading.Thread(target=handle_client, args=(conn, addr))
            t.daemon = True; t.start()
            print(f"[Server] Active threads: {threading.active_count()-1}")

tcp_server()
```

Output:

```
[Server] Listening on 127.0.0.1:65432 [Server] Connection from ('127.0.0.1', 54823) [Server] Active
threads: 1 [Server] From ('127.0.0.1', 54823): Hello Server! [Server] From ('127.0.0.1', 54823): How
are you? [Server] From ('127.0.0.1', 54823): quit [Server] ('127.0.0.1', 54823) disconnected.
```

Note: Multi-threaded server — each client handled in separate thread. Run before client program.

Q20. Write a program to display IP address, port number.

Program:

```
import socket

print("=" * 48)
print("    IP ADDRESS & PORT INFORMATION")
print("=" * 48)

hostname = socket.gethostname()
local_ip = socket.gethostbyname(hostname)
print(f"Hostname   : {hostname}")
print(f"Local IP    : {local_ip}")
print(f"Loopback    : 127.0.0.1")

# Well-known ports
ports = {
    "FTP":21, "SSH":22, "SMTP":25, "DNS":53,
    "HTTP":80, "HTTPS":443, "MySQL":3306, "Flask":5000
}

print("\nWell-known Port Numbers:")
for name, p in ports.items():
```

```

    print(f" {name:<8}: {p}")

# Service lookup
print("\nService lookup:")
for svc in ["http", "https", "ftp", "smtp"]:
    p = socket.getservbyname(svc)
    print(f" {svc.upper():<6} -> Port {p}")

# Bind to dynamic port
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(('', 0))
print(f"\nDynamic port assigned: {sock.getsockname()[1]}")
sock.close()

# External IP (no data sent)
try:
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 80))
    print(f"External IP: {s.getsockname()[0]}")
    s.close()
except:
    print("External IP: (offline)")

```

Output:

```

===== IP ADDRESS & PORT INFORMATION
=====
Hostname : DESKTOP-ABC123 Local IP : 192.168.1.105
Loopback : 127.0.0.1 Well-known Port Numbers: FTP : 21 SSH : 22 HTTP : 80 HTTPS : 443 MySQL : 3306
Service lookup: HTTP -> Port 80 HTTPS -> Port 443 FTP -> Port 21 SMTP -> Port 25 Dynamic port
assigned: 51234 External IP: 192.168.1.105

```

Q21. Write a program to display URL information.

Program:

```

import urllib.parse

def analyze_url(url):
    print(f"\nURL: {url}")
    print("-" * 55)
    p = urllib.parse.urlparse(url)
    print(f" Scheme   : {p.scheme}")
    print(f" Host     : {p.hostname}")
    print(f" Port     : {p.port or 'default'}")
    print(f" Path     : {p.path or '/'}")
    print(f" Query    : {p.query or 'none'}")
    print(f" Fragment : {p.fragment or 'none'}")
    if p.query:
        params = urllib.parse.parse_qs(p.query)
        print(" Params  :")
        for k, v in params.items():
            print(f"      {k} = {v[0]}")

urls = [
    "https://www.google.com/search?q=python+socket&lang=en#results",
    "http://api.example.com:8080/v1/users?id=42&format=json",
    "ftp://ftp.gnu.org/gnu/python/",
]
for u in urls:
    analyze_url(u)

# Encode / Decode

```

```

text = "Python Sockets & Networking = Awesome!"
enc = urllib.parse.quote(text)
print(f"\nEncoded : {enc}")
print(f"Decoded : {urllib.parse.unquote(enc)}")

# Build URL
q = urllib.parse.urlencode({"q": "python", "page": "1"})
built = urllib.parse.ParseResult("https", "mysite.com", "/search", "", q, "top")
print(f"Built URL: {urllib.parse.urlunparse(built)}")

```

Output:

```

URL: https://www.google.com/search?q=python+socket=&en#results
----- Scheme : https Host : www.google.com Port :
default Path : /search Query : q=python+socket=&en Fragment : results Params : q = python socket lang
= en Encoded : Python%20Sockets%20%26%20Networking%20%3D%20Awesome%21 Decoded : Python Sockets &
Networking = Awesome! Built URL: https://mysite.com/search?q=python&page;=1#top

```

Note: urllib.parse.urlparse() splits URL into scheme, host, port, path, query, fragment components.

Q22. Write a program to create application to sending E-mail.

Program:

```

import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
import os

def send_email(sender, password, receiver, subject, body, attach=None):
    msg = MIMEMultipart()
    msg['From'] = sender
    msg['To'] = receiver
    msg['Subject'] = subject
    msg.attach(MIMEText(body, 'plain'))

    if attach and os.path.exists(attach):
        with open(attach, "rb") as f:
            part = MIMEBase('application', 'octet-stream')
            part.set_payload(f.read())
            encoders.encode_base64(part)
            part.add_header('Content-Disposition',
                           f'attachment; filename={os.path.basename(attach)}')
            msg.attach(part)
        print(f"Attachment: {attach}")

    try:
        with smtplib.SMTP('smtp.gmail.com', 587) as server:
            server.ehlo()
            server.starttls() # TLS encryption
            server.ehlo()
            server.login(sender, password)
            server.sendmail(sender, receiver, msg.as_string())
            print(f"Email sent to {receiver}!")
        return True
    except smtplib.SMTPAuthenticationError:
        print("Auth failed! Use Gmail App Password.")
        return False
    except Exception as e:
        print(f"Error: {e}")
        return False

```



```
# Configuration
SENDER = "your_email@gmail.com"      # Your Gmail
PASSWORD = "your_app_password"        # 16-char App Password
RECEIVER = "receiver@gmail.com"

body = (
    "Dear Student,\n\n"
    "This email was sent using Python smtplib module.\n"
    "Topic: Unit 1 - Networking in Python\n\n"
    "Regards,\nPython Program"
)

result = send_email(SENDER, PASSWORD, RECEIVER,
    "Python Email - Advanced Programming", body)
print("Status:", "SUCCESS" if result else "FAILED")
```

Output:

Email sent to receiver@gmail.com! Status: SUCCESS

Note: Gmail setup: Google Account > Security > App Passwords > generate 16-char password. Never hardcode credentials in production.
